

**Leveraging Data Analytics for Student Success: Early Risk Detection and
Intervention System for Delayed Graduation**

Del Rosario, Micaella C.

Jimeno, John Harvey O.

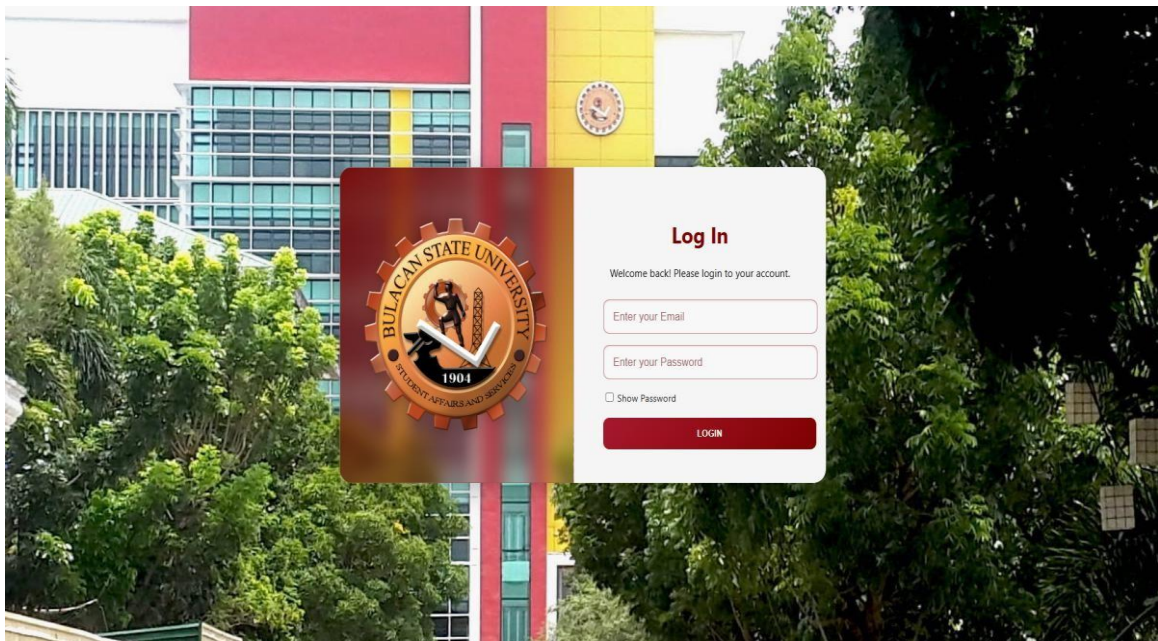
Montallana, Khert Aldrin S.

Navia, Lorenz Grace S.

Salapong, Eddrick Lloyd B.

**BULACAN STATE UNIVERSITY
College of Information and Communications Technology**

November 2025

Figure 86*Local Admin Login*

The login process enhances security and accountability, preventing unauthorized access and maintaining the confidentiality of academic records. The intuitive design also promotes usability, enabling administrators to efficiently log in and begin their work without unnecessary delays.

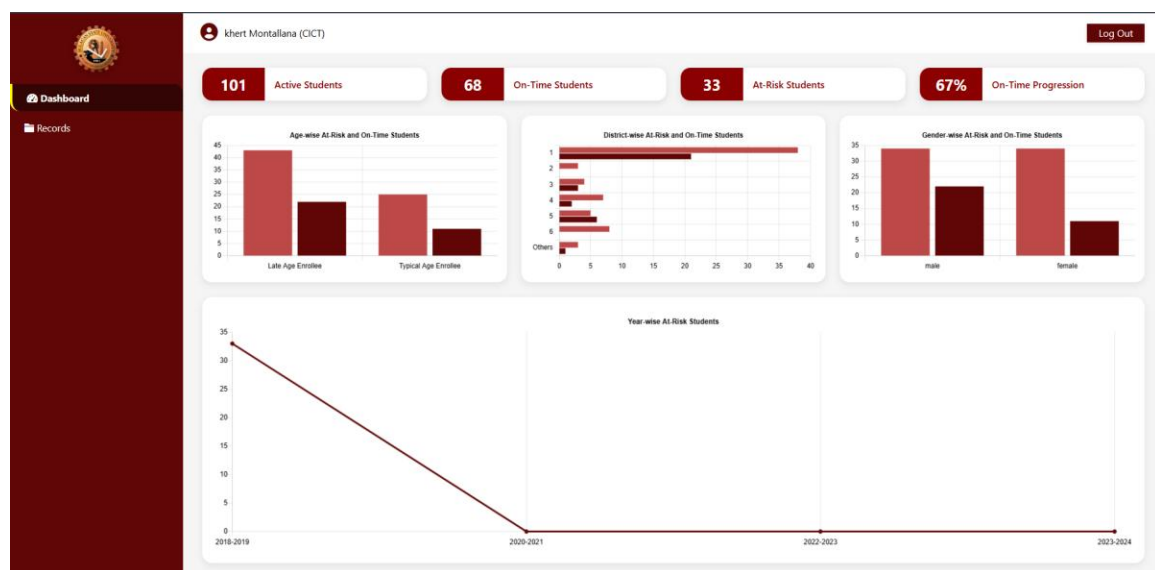
The Local Admin Dashboard is a central feature of the early risk detection and intervention system, designed to provide university administrators with a comprehensive and actionable overview of student performance and progression. Upon logging in, administrators are presented with a series of interactive charts and key performance indicators that collectively support the identification and management of students at risk of delayed graduation.

Figure 87, The dashboard is made up of several distinct visualizations, each serving a specific analytical purpose. The first chart shows the total number of active students, offering a snapshot of current enrollment and participation within the system. Right beside it is a bar chart that illustrates the distribution of at-risk students according to predefined risk categories, allowing administrators to gauge the prevalence and severity of graduation

risks among students. Another key chart displays the count of students who are progressing on time, providing context that shows if interventions are effective. Additionally, the dashboard features a chart that tracks the rate of intervention engagement, highlighting how many at-risk students have received targeted support measures. A final chart plots the progression trends of students over several academic years, enabling continuous analysis of institutional performance and the impact of implemented interventions.

Figure 87

Local Admin Dashboard



This data-driven overview aids in monitoring trends, identifying patterns, and prioritizing interventions. By offering a real-time snapshot of student outcomes, the dashboard empowers administrators to make informed decisions and allocate resources effectively to support students at risk of delayed graduation.

In figure 88, The Local Admin Records is shown. Its component is a key element of the predictive analytics system for student graduation management, serving as the primary interface through which administrative users can access, review, and manage detailed student information.

Figure 88*Local Admin Records*

Student Number	A.Y. Started	Student Name	Course	1st Yr 1st Sem	1st Yr 2nd Sem	2nd Yr 1st Sem	2nd Yr 2nd Sem	Status	Major Risk Factor	Suggested Intervention	Actions
2018107837	2018-2019	GALANG, Kyle Justin	BLIS	Major GWA: 2.35 GE GWA: 1.88	Major GWA: 2.29 GE GWA: 1.68	Major GWA: 2.79 GE GWA: 2.25	Major GWA: 2.80 GE GWA: 1.00	At-Risk	second_year_first_sem_major	Provide tutoring for major subjects in 2nd year 1st semester	Archive
Birthdate: 1999-10-10 Gender: male Citizenship: Filipino Home Address: Malolos, Bulacan Religion: ROMAN CATHOLIC Created By Email: admin1@email.com Created At: 2025-10-26 23:05:24 Updated At: 2025-10-26 23:05:24											
2018107856	2018-2019	DIZON, Patricia Ann Piozon	BLIS	Major GWA: 2.00 GE GWA: 1.78	Major GWA: 2.14 GE GWA: 1.75	Major GWA: 2.30 GE GWA: 1.50	Major GWA: 2.34 GE GWA: 1.50	At-Risk	second_year_first_sem_major	Provide tutoring for major subjects in 2nd year 1st semester	Archive
2018107859	2018-2019	CRISTOBAL, Crisha Mae O.	BLIS	Major GWA: 1.83 GE GWA: 1.60	Major GWA: 1.64 GE GWA: 1.66	Major GWA: 1.80 GE GWA: 1.50	Major GWA: 1.80 GE GWA: 1.25	On-Time	second_year_second_sem_major	No Intervention Needed	Archive
2018107861	2018-2019	ACCAD, Ashly Galle Catara	BLIS	Major GWA: 2.04 GE GWA: 1.66	Major GWA: 2.36 GE GWA: 1.91	Major GWA: 2.13 GE GWA: 1.50	Major GWA: 2.26 GE GWA: 1.50	On-Time	second_year_second_sem_major	No Intervention Needed	Archive
2018107863	2018-2019	SANCHEZ, Ivy Mendoza	BLIS	Major GWA: 2.00 GE GWA: 1.66	Major GWA: 2.02 GE GWA: 1.96	Major GWA: 1.97 GE GWA: 1.50	Major GWA: 1.97 GE GWA: 1.50	On-Time	second_year_second_sem_major	No Intervention Needed	Archive

The use of SHAP framework in the Local Admin Records component offers an interpretable and data-driven basis on the creation of specific academic interventions. Once a student is determined to be at risk of delayed graduation by the predictive model, SHAP measures the role of each input feature, i.e. semester grades, age group, gender, etc. in the prediction and finds out which risk factor has the greatest impact on the prediction. Such information will allow local administrators to see the exact cause of each prediction and implement a related intervention that will directly respond to the observed risk. In this case, when the big risk factor is linked with poor academic performance during a specific semester or subject area, the system suggests tutoring assistance depending on the specific subjects to enhance competency and retention. On the other hand, in case the risk factor relates to demographic factors like age group or gender, the system would propose guidance counseling as a way of offering psychosocial or motivational assistance that can probably improve academic engagement. Combining SHAP-based explanations with the systematic presentation of the student information, the Local Admin Records component facilitates the transparency of the decision-making process and allows the administrator to introduce the accurate and evidence-based interventions, which are based on the unique needs of the student. This integration of explicitly artificially intelligent and lengthwise scholarly management makes all institutions more responsive whereby at-risk students are

provided with timely and individualized support to enhance their opportunities of graduating on time.

Figure 89 The Local Admin Input form enables administrators to add or update student information directly within the system. By entering academic details, risk indicators, and intervention notes, users ensure that the database remains accurate and comprehensive.

Figure 89

Local Admin Input

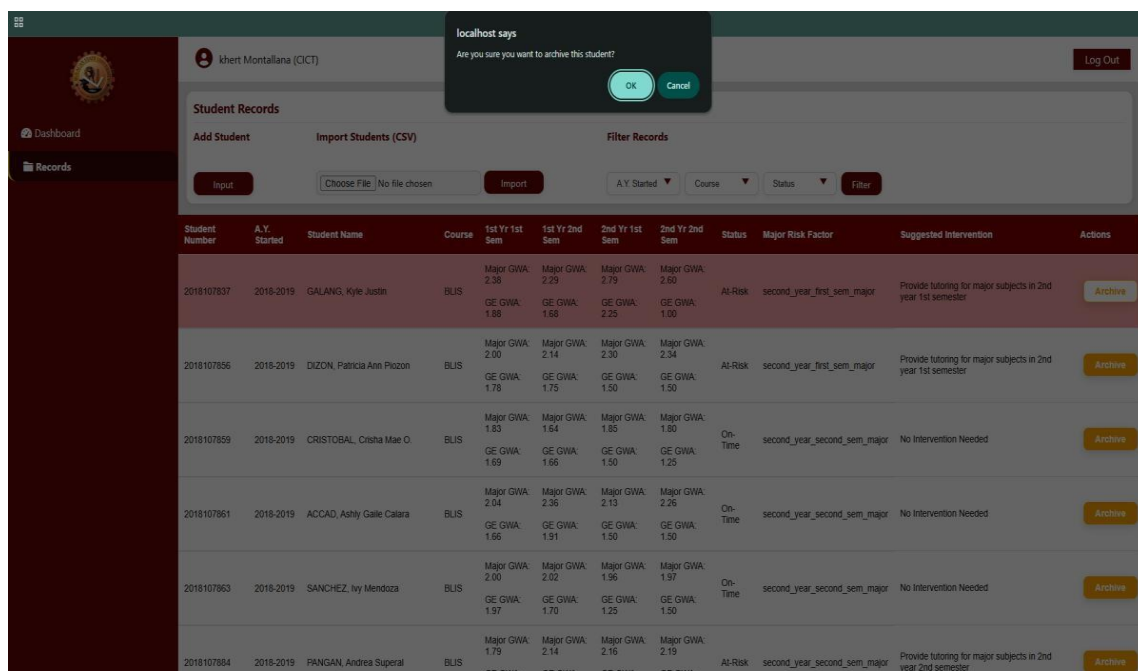
The screenshot displays the 'Local Admin Input' form within the 'khert Montallana (CICT)' system. The form is a modal window that allows administrators to add or update student information. It includes the following sections:

- Student Details:** Fields for Student Number (2020106354), Academic Year Started (2020-2021), Student Name (Harvey Jimenez), Birthdate (10/11/2002), Gender (Male), Citizenship (Filipino), Religion (Catholic), Home Address (guiguinto, Bulacan), and County (Bulacan).
- Grades:** Fields for First Year First Sem - Major GWA (1.68), First Year First Sem - GE GWA (1.36), First Year Second Sem - Major GWA (1.23), First Year Second Sem - GE GWA (1.34), Second Year First Sem - Major GWA (1.65), Second Year First Sem - GE GWA (1.45), Second Year Second Sem - Major GWA (1.45), and Second Year Second Sem - GE GWA (1.44).
- Major Risk Factor, Suggested Intervention, and Actions:** A table with columns for Major Risk Factor, Suggested Intervention, and Actions. The table contains several rows of data, including 'second_year_first_sem_major' and 'second_year_second_sem_major' with suggested interventions like 'Provide tutoring for major subjects in 2nd year 1st semester' and 'No Intervention Needed'.

The background shows a dashboard with a sidebar containing 'Dashboard' and 'Records' links. The main content area displays a table of student records with columns for Student Number, A.Y. Started, Student Name, and various academic metrics.

This component supports both manual entry and bulk imports, accommodating various data management needs. The input form is designed for ease of use, reducing the risk of errors and improving the efficiency of administrative operations related to student success tracking

Figure 90, The Local Admin Archive functions as a secure, organized repository for historical student records, supporting the university's commitment to evidence-based monitoring and intervention. This component provides administrative users with comprehensive access to archived data, including academic metrics, graduation risk indicators, and intervention outcomes for each student.

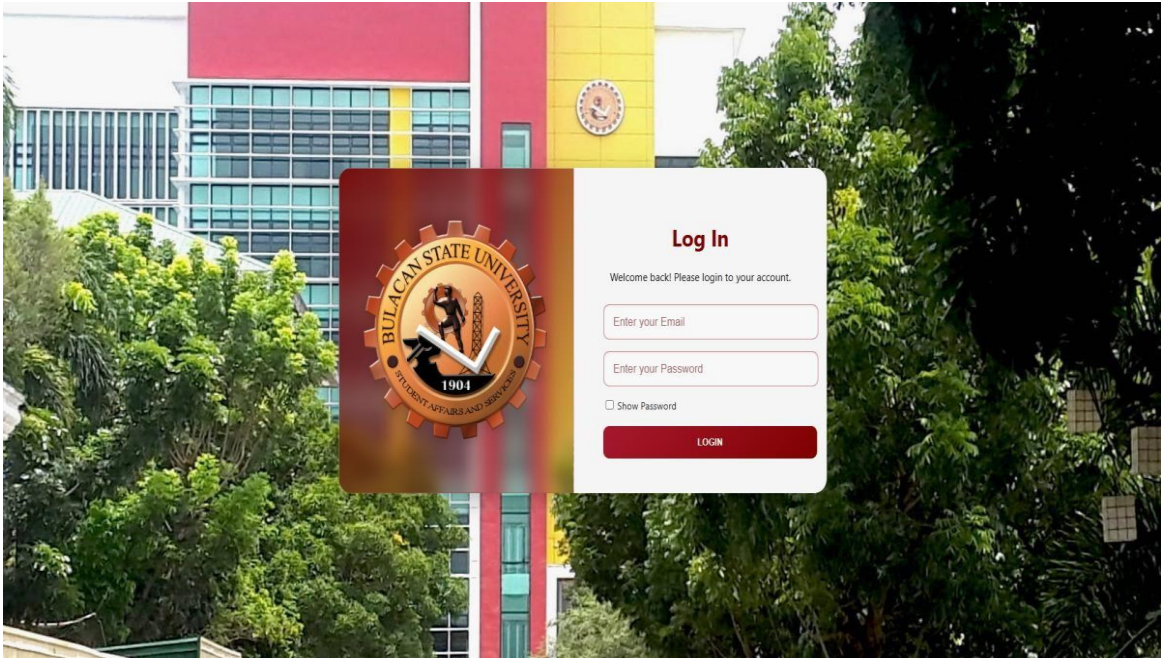
Figure 90*Local Admin Archive*


The screenshot shows a web application interface for 'Local Admin Archive'. At the top, a user 'khert Montallana (CICT)' is logged in. A modal dialog is open with the text 'localhost says Are you sure you want to archive this student?' and 'OK' and 'Cancel' buttons. Below the modal, there are tabs for 'Add Student', 'Import Students (CSV)', and 'Filter Records'. The 'Filter Records' tab is active, showing a table of student records. The table has columns for Student Number, A.Y. Started, Student Name, Course, 1st Yr 1st Sem, 1st Yr 2nd Sem, 2nd Yr 1st Sem, 2nd Yr 2nd Sem, Status, Major Risk Factor, Suggested Intervention, and Actions. The table contains six rows of student data.

Student Number	A.Y. Started	Student Name	Course	1st Yr 1st Sem	1st Yr 2nd Sem	2nd Yr 1st Sem	2nd Yr 2nd Sem	Status	Major Risk Factor	Suggested Intervention	Actions
2018107837	2018-2019	GALANG, Kyle Justin	BLIS	Major GWA: 2.36 GE GWA: 1.88	Major GWA: 2.29 GE GWA: 1.66	Major GWA: 2.75 GE GWA: 2.25	Major GWA: 2.60 GE GWA: 1.90	At-Risk	second_year_first_sem_major	Provide tutoring for major subjects in 2nd year 1st semester	Archive
2018107856	2018-2019	DIZON, Patricia Ann Picozon	BLIS	Major GWA: 2.00 GE GWA: 1.78	Major GWA: 2.14 GE GWA: 1.75	Major GWA: 2.30 GE GWA: 1.50	Major GWA: 2.34 GE GWA: 1.50	At-Risk	second_year_first_sem_major	Provide tutoring for major subjects in 2nd year 1st semester	Archive
2018107859	2018-2019	CRISTOBAL, Crisha Mae O.	BLIS	Major GWA: 1.83 GE GWA: 1.69	Major GWA: 1.64 GE GWA: 1.66	Major GWA: 1.85 GE GWA: 1.50	Major GWA: 1.80 GE GWA: 1.25	On-Time	second_year_second_sem_major	No Intervention Needed	Archive
2018107861	2018-2019	ACCAD, Ashy Gale Calara	BLIS	Major GWA: 2.04 GE GWA: 1.66	Major GWA: 2.36 GE GWA: 1.91	Major GWA: 2.13 GE GWA: 1.50	Major GWA: 2.26 GE GWA: 1.50	On-Time	second_year_second_sem_major	No Intervention Needed	Archive
2018107863	2018-2019	SANCHEZ, Ivy Mendoza	BLIS	Major GWA: 2.00 GE GWA: 1.97	Major GWA: 2.02 GE GWA: 1.70	Major GWA: 1.96 GE GWA: 1.25	Major GWA: 1.97 GE GWA: 1.50	On-Time	second_year_second_sem_major	No Intervention Needed	Archive
2018107884	2018-2019	PANGAN, Andrea Superal	BLIS	Major GWA: 1.79 GE GWA: 1.79	Major GWA: 2.14 GE GWA: 1.79	Major GWA: 2.16 GE GWA: 1.79	Major GWA: 2.19 GE GWA: 1.79	At-Risk	second_year_second_sem_major	Provide tutoring for major subjects in 2nd year 2nd semester	Archive

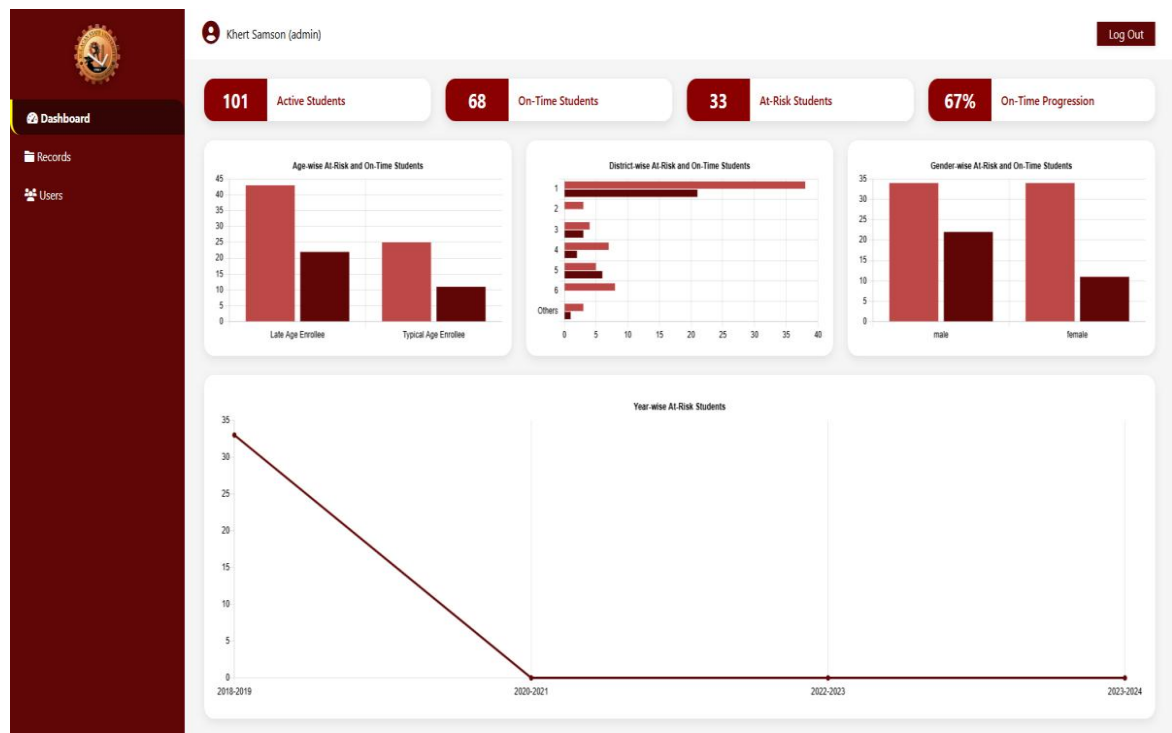
By retaining these records within the system, the archive facilitates longitudinal analysis, enabling administrators to trace individual student trajectories and evaluate the impact of previous support strategies over time. The intuitive interface allows for efficient retrieval, filtering, and export of records, ensuring that critical information remains readily available for ongoing review and assessment. Furthermore, the archive's integration with predictive analytics enhances its methodological value, as historical patterns can inform the development and refinement of risk models and intervention protocols. In this way, the Local Admin Archive not only safeguards institutional memory but also empowers stakeholders to make data-driven decisions that advance student success and timely graduation.

Figure 91, The Registrar Login interface provides a secure access point for university registrars, ensuring that only authorized personnel can manage and analyze student data. Implementing robust authentication mechanisms, this component protects sensitive information and upholds data privacy standards

Figure 91*Registrar Login*

The login screen is designed for ease of use, allowing registrars to quickly enter their credentials and access the dashboard. By restricting access to accredited users, the system maintains the integrity of student records and supports compliance with institutional data governance policies.

Figure 92, The Registrar Dashboard provides a high-level, analytics-driven overview of student progression and graduation risk, utilizing visualizations and key performance indicators similar to those found in the Local Admin Dashboard. This interface combines critical data, such as the number of active students, at-risk students, intervention engagement rates, and progression trends, all into a format tailored for analysis and strategic planning. While the dashboard's design and contents are largely consistent with those of the local admin system, its scope of relevance is expanded to all of the data from other colleges that helps with their responsibilities in reporting, compliance, and institutional coordination.

Figure 92*Registrar Dashboard*

By presenting consolidated insights and historical patterns, the Registrar Dashboard enables registrars to monitor the effectiveness of interventions, identify emerging challenges, and contribute to the development of data-informed policies that promote timely graduation. The seamless integration of predictive analytics into the dashboard supports the study's objective of leveraging data for proactive student support.

Figure 93, The Registrar Records component is similar to the Local Admin Records in both structure and function, serving as a central repository for detailed student information accessible by authorized registrar personnel. This section presents a tabular view of individual student profiles, academic performance indicators, and intervention statuses, enabling efficient tracking and management of graduation risks. While the user interface closely mirrors those of the local admin system, the registrar's access is vastly greater than local admins as their scope is the entirety of the university, reflecting their bigger responsibility across everything.

Figure 93*Registrar Records*

Student Number	A.Y. Started	Student Name	Course	1st Yr 1st Sem	1st Yr 2nd Sem	2nd Yr 1st Sem	2nd Yr 2nd Sem	Status	Major Risk Factor	Suggested Intervention	Actions
2018100313	2018-2019	ABACAN, Hans Joseph Sacdalan	BSIT	Major GWA: 2.05 GE GWA: 1.84	Major GWA: 3.44 GE GWA: 1.94	Major GWA: 3.15 GE GWA: 1.65	Major GWA: 2.67 GE GWA: 1.60	At-Risk	second_year_first_sem_major	Provide tutoring for major subjects in 2nd year 1st semester	Archive, Update, Delete
Birthdate: 1970-01-01 Gender: male Citizenship: Filipino Home Address: Pullian, Bulacan Religion: ROMAN CATHOLIC Created By Email: admin1@email.com Created At: 2025-10-26 23:05:35 Updated At: 2025-10-26 23:05:35											
2018100316	2018-2019	ABADILLA, Albert D.	BSIT	Major GWA: 1.95 GE GWA: 1.25	Major GWA: 2.05 GE GWA: 1.19	Major GWA: 1.75 GE GWA: 2.00	Major GWA: 1.92 GE GWA: 2.40	On-Time	second_year_first_sem_major	No intervention Needed	Archive, Update, Delete
2018100352	2018-2019	ABANGGAN, Ej Jalen Llantero	BSIT	Major GWA: 2.99 GE GWA: 3.16	Major GWA: 2.92 GE GWA: 2.30	Major GWA: 2.12 GE GWA: 1.55	Major GWA: 2.77 GE GWA: 1.70	At-Risk	second_year_second_sem_major	Provide tutoring for major subjects in 2nd year 2nd semester	Archive, Update, Delete

The SHAP method is utilized to improve the interpretability and explainability of the developed Graduation Prediction System to provide evidence-based academic intervention. SHAP is also based on theory and differs in that it explains model predictions through the use of quantitative contribution values whereby each input feature is attributed with a measurement of contribution for the output of the model (like academic performance, gender, district, and age group). By applying the treeexplainer, which is designed to handle tree-based models such as a random forest, the system calculates the degree to which each feature raises or lowers the probability of a student being considered as being at-risk. The feature which has the largest magnitude of SHAP value is considered the major risk factor and this is a valuable clue of the most significant determinant of delayed graduation. By recognizing the major risk factor of a student, the registrar can initiate early intervention through targeted tutoring sessions. This allows the student to strengthen weak academic areas and reduce the likelihood of delayed graduation. This interpretative power allows the university's registrar to grasp the result as well as the reasoning behind this outcome being projected. Moreover, the superior search and filter capabilities of the component facilitate systematic review and monitoring enabling the registrars to effectively recognize at-risk students, authenticate academic record, and manage the primary risk factor affecting each case. The system enhances the overall

institutional ability to make data-active decisions and improve on student retention and graduation rates by introducing predictive analytics into the operational processes of the registrar. There have been other recent studies that applied SHAP to gain insight into educational intervention systems (Guan et al., 2025), where SHAP-based explanations served to find relevant predictors which affect student performance and to guide academic support interventions.

Figure 94, The Registrar Input component streamlines the process of adding new student records to the system, supporting the timely and accurate expansion of datasets. Through a structured input form, registrar personnel can enter key student details, academic indicators, and initial intervention notes. The interface is designed for usability and data consistency, with clearly labeled fields and validation protocols that minimize entry errors

Figure 94

Registrar Input

The screenshot shows a web application interface for a registrar. A modal form titled "Student Details" is open, allowing the user to add a new student record. The form includes the following fields:

- Student Number: 2020100304
- Academic Year Started: 2020-2021
- Student Name: Micaela C. Del Rosario
- Birthdate: 12/12/2002
- Gender: Female
- Religion: Christian
- Citizenship: Filipino
- Home Address: Pander, Bulacan
- Course: BSIT - Bachelor of Science in Informatic

Below these fields are input boxes for Grade Point Averages (GWA):

- First Year First Sem - Major GWA: 1.65
- First Year First Sem - GE GWA: 1.45
- First Year Second Sem - Major GWA: 1.54
- First Year Second Sem - GE GWA: 1.22
- Second Year First Sem - Major GWA: 1.67
- Second Year First Sem - GE GWA: 1.34
- Second Year Second Sem - Major GWA: 1.37
- Second Year Second Sem - GE GWA: 1.34

The form has "Confirm" and "Cancel" buttons at the bottom. In the background, a table of student records is visible. The table has columns for Student Number, A.Y. Started, Student Name, Major GWA, GE GWA, Major GWA, GE GWA, At-Risk, second_year_first_sem_major, and Actions. The table contains several rows of student data, including ABACAN, HANS JOSEPH S., ABADILLA, ALBERT D., ABANGGAN, EJ JALEN LANTERO, and ABAROA, DAN CARLO V.

This functionality is critical for supporting the ongoing collection of comprehensive data required for effective predictive analytics. By enabling rapid incorporation of new cases, the input component enhances the responsiveness of the early risk detection system and ensures that all relevant student information is available for analysis and intervention planning.

Figure 95, The Registrar Update feature facilitates the ongoing maintenance and refinement of student records, allowing authorized users to amend existing entries in response to new information or changes in status. The update interface presents a focused form view, where registrar personnel can review and modify academic metrics, personal details, or intervention outcomes as needed.

Figure 95

Registrar Update

The screenshot displays a 'Registrar Update' form with two main sections: 'Student Details' and 'Grades'.

Student Details:

- Student Number:** 2021106354
- Academic Year Started:** 2021-2022
- Student Name:** Micaela C. Del Rosario
- Birthdate:** 01/02/2003
- Gender:** Female
- Citizenship:** Filipino
- Religion:** Christian
- Home Address:** Paraiso, Bulacan
- Course:** BSIT - Bachelor of Science in Informatics

Grades:

First Year First Sem - Major GWA:	First Year First Sem - GE GWA:
1.28	1.11
First Year Second Sem - Major GWA:	First Year Second Sem - GE GWA:
1.32	1.45
Second Year First Sem - Major GWA:	Second Year First Sem - GE GWA:
1.43	1.22
Second Year Second Sem - Major GWA:	Second Year Second Sem - GE GWA:
1.54	1.34

At the bottom of the form, there are two buttons: 'Confirm' and 'Cancel'.

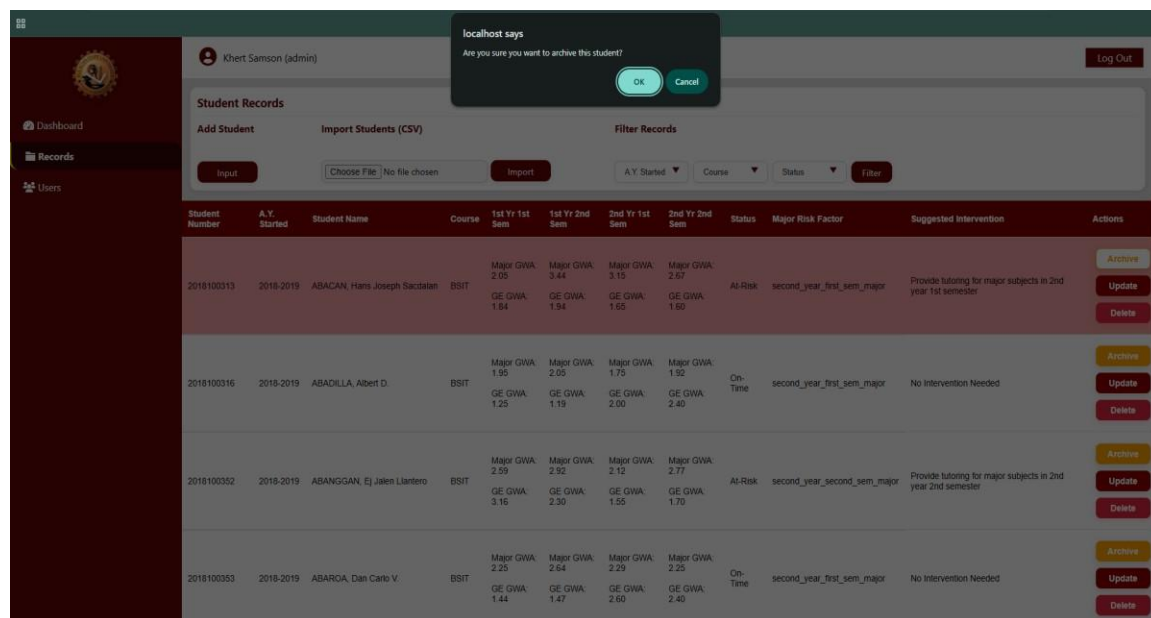
This capability is essential for upholding the accuracy and relevance of the dataset, which helps with the reliability of predictive models and risk assessments. By supporting iterative updates to student records, the component helps ensure that the university's analytics-driven approach to graduation risk remains updated with present situations.

Figure 96, The Registrar Archive component serves as a secure long term storage of student records, ensuring that essential data is preserved for institutional review and future analysis. Within this interface, authorized registrar personnel can view a comprehensive list of student records, including academic performance indicators, progression status, and intervention history. The archiving feature is accompanied by a

confirmation dialog, which prompts users to verify their intent before moving records into the archive, thereby safeguarding against accidental data mismanagement.

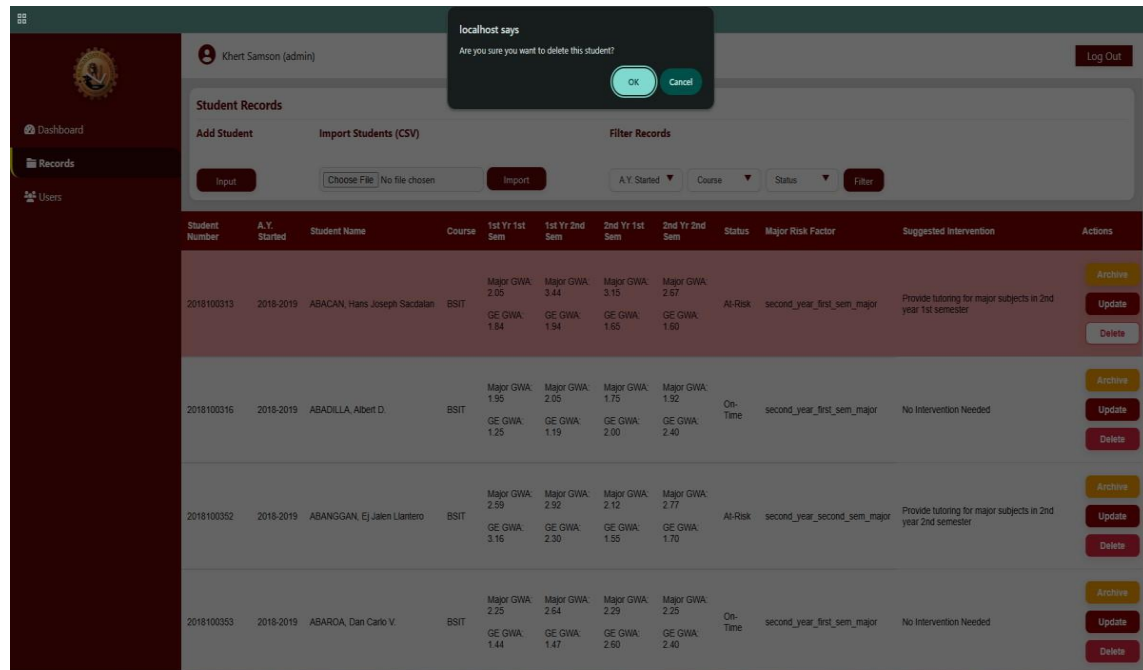
Figure 96

Registrar Archive



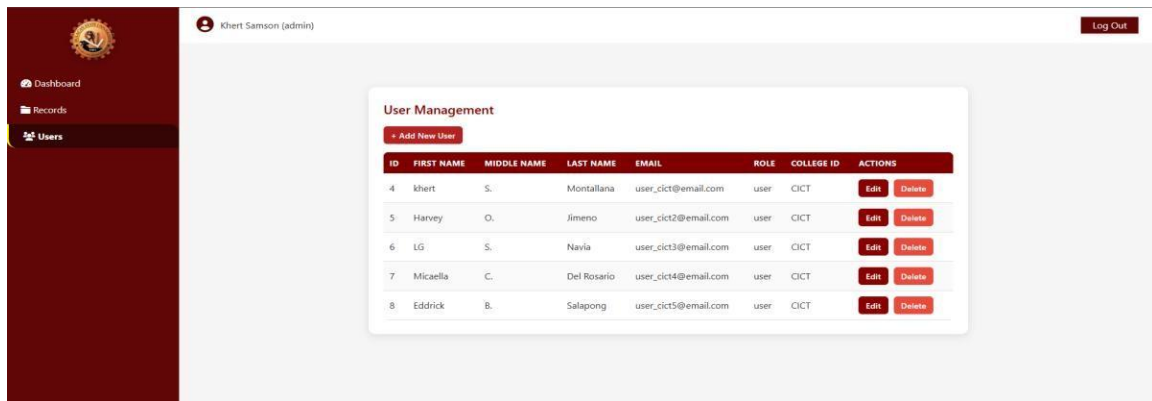
By retaining historical records in an accessible format, this component supports longitudinal studies of student outcomes and the continuous refinement of predictive models. Its integration within the system underscores a commitment to data-driven decision making, enabling registrars to reference archived cases when evaluating the efficacy of graduation interventions or developing new support strategies.

Figure 97, The Registrar Student Record Delete component provides registrar personnel with a secure and deliberate mechanism for removing student records from the system's database. This feature presents a tabular list of student information, including academic metrics and intervention status, alongside action buttons that facilitate record management. Upon initiating a deletion, users are prompted with a confirmation dialog to verify the decision, thereby minimizing the risk of accidental data loss and reinforcing data management accuracy.

Figure 97*Registrar Student Record Delete*

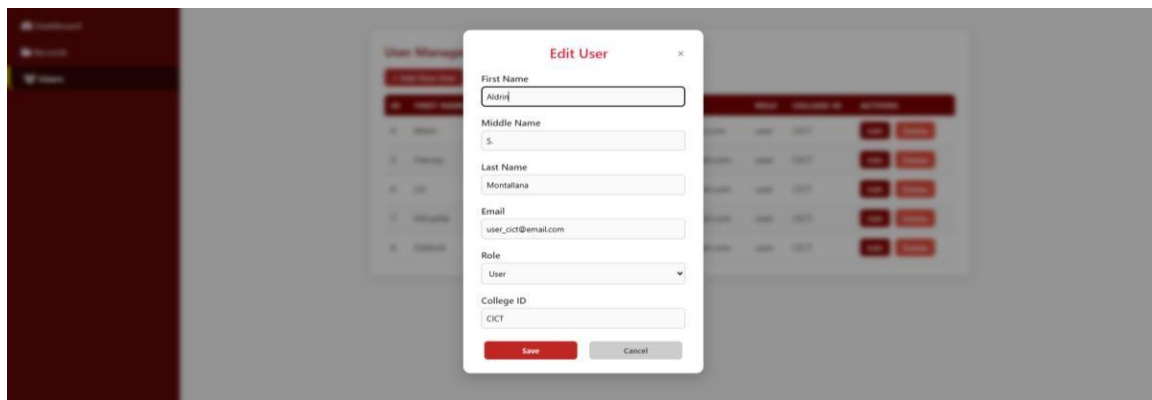
By enforcing a clear confirmation step, the system upholds best practices in data governance and administrative accountability. The ability to delete outdated or mislabeled records ensures that the database remains relevant and focused on current students, which is essential for maintaining the integrity of predictive analytics and supporting data driven decision making within the university.

Figure 98, The Registrar User Management component is designed to support the effective oversight and administration of system users by registrar staff. Through an organized interface, user accounts are displayed with essential details such as names, email addresses, roles, and affiliated colleges, enabling quick review and management of user access. Action buttons provide the ability to edit or remove user accounts, facilitating responsive updates to permissions and roles as university needs evolve.

Figure 98*Registrar User Management*

This feature is integral to maintaining system security and operational integrity, as it allows registrars to ensure that only authorized personnel can interact with sensitive student data and analytics tools. By streamlining user management processes, the system supports compliance with institutional policies and fosters a controlled environment for collaborative student success interventions.

Figure 99, The Registrar User Edit can be utilized to edit information of the users including their name, email, role and College ID. Through the input form the registrar can modify existing details. Once the changes are made, Clicking save commits the changes to be updated in the database, ensuring user records remain up-to-date.

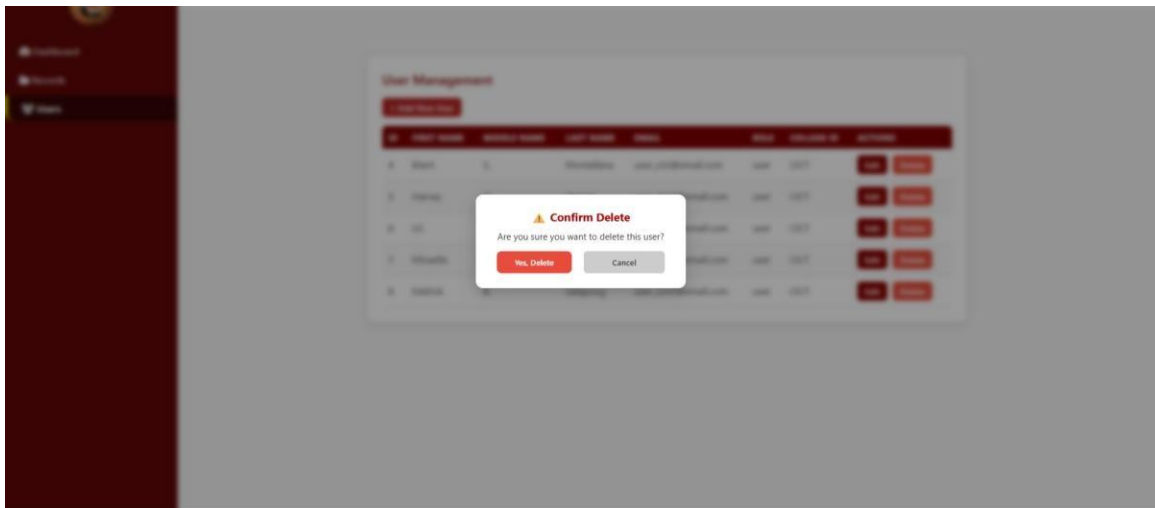
Figure 99*Registrar User Edit*

This feature is important for maintaining accurate and reliable information of the users, especially when roles and associations of the institution changes. This allows for an efficient management of user credentials.

Figure 100, The user delete function allows the registrar to efficiently remove the user from the record of the system. This is only valid so that only active user accounts remain in the database. The registrar can choose to confirm the deletion permanently or to keep it active and accessible for the institution.

Figure 100

Registrar User Delete



This feature is essential for security, accuracy and efficiency of the system management. It gives control over which account remains and which are to be removed. It also supports data integrity as it keeps the database away from unauthorized and duplicated account.

```

<?php
function getDbConnection()
{ static $conn;
  if ($conn === null) {
    $host = 'localhost';
    $db =
'bulsu_student_delay_prediction_system
_database';
    $user = 'root';
    $pass = "";
    $conn = new mysqli($host, $user,
$pass, $db);
    if ($conn->connect_error)
    { die("Connection failed:
    "
    $conn->connect_error);
    return new
Exception("Connection failed.");
    }
  }
  return $conn;
}
*****
from flask import Flask, request, jsonify
import joblib
import pandas as pd
import os
from datetime import datetime
import shap

app = Flask(__name__)

# Load model
model_path =
os.path.join(os.path.dirname(__file__),
"rf_model.pkl")
model = joblib.load(model_path)

# Expected columns for the
model
expected_columns = [

'first_year_first_sem_major',
'first_year_second_sem_major',

'second_year_first_sem_major',
'second_year_second_sem_major',

```

```

'first_year_first_sem_ge',
'first_year_second_sem_ge',
'second_year_first_sem_ge',
'second_year_second_sem_ge',

'age_group_Late Age
Enrollee', 'age_group_Typical Age
Enrollee',

'gender_Female',
'gender_Male',

'district_District 1',
'district_District 2', 'district_District 3',
'district_District 4',
'district_District 5', 'district_District 6',
'district_Outside Bulacan'
]

# NOTE: The explainer only
needs to be created once for the model.
explainer =
shap.TreeExplainer(model)

@app.route("/predict",
methods=["POST"])
def predict():
    data = request.get_json()
    birthdate = data.get('birthdate')

    ay_started =
data.get('ay_started')

# --- AGE COMPUTATION --
-

if birthdate and ay_started:
    birthdate_dt =
datetime.strptime(birthdate, "%d/%m/%Y")

    print(birthdate_dt)

# Handle AY like "2022-
2023" or full date

```

```

        if "-" in ay_started and
len(ay_started) == 9:
            start_year =
int(ay_started.split('-')[0])
            ay_dt =
datetime(start_year, 8, 1) # Assume Aug
1 start
        else:
            ay_dt =
datetime.strptime(ay_started,
"%d/%m/%Y")

            age = round((ay_dt -
birthdate_dt).days / 365.25, 2)

            if age <= 19.5:
                data['age_group_Typical
Age Enrollee'] = 1
                data['age_group_Late
Age Enrollee'] = 0
            else:
                data['Age Group_Typical
Age Enrollee'] = 0
                data['Age Group_Late
Age Enrollee'] = 1
            else:
                data['Age Group_Typical
Age Enrollee'] = 0
                data['Age Group_Late Age
Enrollee'] = 0

            # --- ENCODINGS ---
            gender = data.get('gender',
").lower()
            data['Gender_F'] = 1 if gender
== 'female' else 0
            data['Gender_M'] = 1 if gender
== 'male' else 0

            address = data.get('address',
").lower()
            district_mapping =
            { 'district 1': ['malolos',
'bulakan', 'calumpit', 'hagonoy',
'paombong', 'pulilan'],
            'district 2': ['baliwag city',
'bustos', 'plaridel'],
            'district 3': ['doña remedios
trinidad', 'san ildefonso', 'san miguel',
'san rafael'],
            'district 4': ['meycauayan',
'marilao', 'obando'],
            'district 5': ['balagtas',
'bocaue', 'guiguinto', 'pandi'],
            'district 6': ['angat',
'norzagaray', 'sta. maria', 'san jose del
monte']}

            for col in ['Address_District 1',
'Address_District 2', 'Address_District
3',
                        'Address_District 4',
'Address_District 5', 'Address_District
6',
                        'Address_Outside
Bulacan']:
                data[col] = 0

                if 'bulacan' in address:
                    for district, towns in
district_mapping.items():
                        if any(town in address for
town in towns):
                            data[f'Address_{district}'] = 1
                            break
                else:
                    data['Address_Outside Bulacan']
= 1

            # --- PREDICT ---
            row = {col: data.get(col, 0) for
col in expected_columns}
            df = pd.DataFrame([row])
            prediction =
model.predict(df)[0]

            # Major_Predictor

```

```

        shap_values =
explainer.shap_values(df)
        # Get class index
        class_index =
list(model.classes_).index(prediction)

        if isinstance(shap_values, list):
            # choose shap values for the
predicted class
            shap_class_values =
shap_values[prediction]
        else:
            # array form (n_samples,
n_features, n_classes)
            shap_class_values =
shap_values[:, :, class_index]

            # Create DataFrame
            contrib =
pd.DataFrame(shap_class_values,
columns=df.columns)
            # Identify feature with the
largest contribution
            top_feature =
contrib.abs().idxmax(axis=1).iloc[0]

            # --- RETURN FULL INFO --
-
            result =
            { "major_predictor
":
top_feature,
            "prediction": str(prediction),
            }

            # Send all data back to PHP
            return jsonify(result)

            if __name__ == "__main__":
                app.run(host="127.0.0.1",
port=5000, debug=True)
*****
<?php
function
predictStatus($studentDetails)
{

```

```

        try {
            $data =
            [ 'ay_started'
=>
$studentDetails['ay_started'],
            'birthdate' =>
$studentDetails['birthdate'],
            'gender' =>
$studentDetails['gender'],
            'address' =>
$studentDetails['home_address'],

            'first_year_first_sem_major' =>
$studentDetails['first_year_first_sem_m
ajor'],

            'first_year_second_sem_major' =>
$studentDetails['first_year_second_sem_
major'],

            'second_year_first_sem_major' =>
$studentDetails['second_year_first_sem_
major'],

            'second_year_second_sem_major' =>
$studentDetails['second_year_second_se
m_major'],

            'first_year_first_sem_ge'
=>
$studentDetails['first_year_first_sem_ge'
],

            'first_year_second_sem_ge' =>
$studentDetails['first_year_second_sem_
ge'],

            'second_year_first_sem_ge' =>
$studentDetails['second_year_first_sem_
ge'],

            'second_year_second_sem_ge' =>
$studentDetails['second_year_second_se
m_ge']

        ];

        $options =
        [ "http" =>
        [

```

```

        "header" => "Content-
type: application/json\r\n",
        "method" => "POST",
        "content" =>
json_encode($data),
    ],
];

    $context =
stream_context_create($options);
    $result =
file_get_contents("http://127.0.0.1:5000/
predict", false, $context);

    if ($result === false)
    { throw new
Exception("Failed to get prediction from
API");
    }

    return $result;
} catch (Exception $e) {
    // You can customize this
error handling based on your needs
    return
        json_encode([ 'err
or' => true,
'message' => $e-
>getMessage()
]);
}
}
*****
<?php
function
predictStatus($studentDetails)
{
    try {
        $data =
            [ 'ay_started'
=>
$studentDetails['ay_started'],
'birthdate' =>
$studentDetails['birthdate'],
'gender' =>
$studentDetails['gender'],
'address' =>
$studentDetails['home_address'],

```

```

'first_year_first_sem_major' =>
$studentDetails['first_year_first_sem_m
ajor'],

'first_year_second_sem_major' =>
$studentDetails['first_year_second_sem_
major'],

'second_year_first_sem_major' =>
$studentDetails['second_year_first_sem_
major'],

'second_year_second_sem_major' =>
$studentDetails['second_year_second_se
m_major'],

        'first_year_first_sem_ge'
=>
$studentDetails['first_year_first_sem_ge'
],

'first_year_second_sem_ge' =>
$studentDetails['first_year_second_sem_
ge'],

'second_year_first_sem_ge' =>
$studentDetails['second_year_first_sem_
ge'],

'second_year_second_sem_ge' =>
$studentDetails['second_year_second_se
m_ge']
    ];

    $options =
    [ "http" =>
    [
        "header" => "Content-
type: application/json\r\n",
        "method" => "POST",
        "content" =>
json_encode($data),
    ],
    ];

    $context =
stream_context_create($options);

```

```

        $result =
file_get_contents("http://127.0.0.1:5000/
predict", false, $context);

        if ($result === false)
        { throw new
Exception("Failed to get prediction from
API");
        }

        return $result;
    } catch (Exception $e) {
        // You can customize this
error handling based on your needs
        return
            json_encode([ 'err
or' => true,
'message' => $e-
>getMessage()
]);
    }
}
}
*****
<?php

// Enable error reporting for
debugging
// error_reporting(E_ALL);
// ini_set('display_errors', 1);

require_once __DIR___.
'../USERHANDLING/User.php';
require_once __DIR___.
'../predictStatus.php';
session_start();

function getColumnValidator()
{
    return [
        'student_number' =>
function ($val) {
            return
preg_match('/^\d{10}$/', $val);
        },
        'ay_started' => function
($val) {

```

```

            return
preg_match('/^\d{4}-\d{4}$/', $val);
        },
        'student_name' => function
($val)
            return strlen(trim($val)) >
{ 0;
        },
        'birthdate' => function
($val) {
            return
DateTime::createFromFormat('d/m/Y',
$val);
        },
        'gender' => function ($val)
        { return
in_array(strtolower($val), ['male',
'female']);
        },
        'citizenship' => function
($val) {
            return strlen(trim($val)) >
0;
        },
        'home_address' => function
($val)
            return strlen(trim($val)) >
{ 0;
        },
        'religion' => function ($val)
        {
            return strlen(trim($val)) >
0;
        },
        'course_id' => function
($val) {
            require_once __DIR___.
'../DB/getDbConnection.php';
            $db =
getDbConnection();
            $user =
$_SESSION['current_user'];
            if ($user->role ===
'admin') {

```

```

        $stmt = $db-
>prepare("SELECT course_id FROM
courses");
    } else {
        $stmt = $db-
>prepare("SELECT course_id FROM
courses WHERE college_id = ?");
        $stmt-
>bind_param("s", $user->college_id);
    }
    $stmt->execute();
    $result = $stmt-
>get_result();
    $validCourses = [];
    while ($row = $result-
>fetch_assoc()) {
        $validCourses[] =
        $row['course_id'];
    }
    $stmt->close();
    return in_array($val,
    $validCourses);
},
'first_year_first_sem_ge'=>
function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'first_year_second_sem_ge'
=> function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'second_year_first_sem_ge'
=> function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'second_year_second_sem_ge'=>
function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'first_year_first_sem_major'
=> function ($val) {

```

```

        return is_numeric($val)
        && $val >= 0 && $val <= 5.0;
    },
'first_year_second_sem_major'=>
function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'second_year_first_sem_major'=>
function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
},
'second_year_second_sem_major'=>
function ($val) {
    return is_numeric($val)
    && $val >= 0 && $val <= 5.0;
}
];
}

function processBatchRecord()
{
    // Check if File format is
correct.
    $allowedExtensions = ['csv'];
    $fileParts = explode('.',
$_FILES['batch_record']['name']);
    $fileExt =
strtolower(end($fileParts));

    if (!in_array($fileExt,
$allowedExtensions)) {
        throw new
Exception("Invalid File Type.");
    }

    $validRows = [];

    if (($handle =
fopen($_FILES['batch_record']['tmp_na
me'], 'r')) !== false) {
        $header = fgetcsv($handle);

```



```

$header = array_map('trim',
$header);

// Ensure all expected
columns exist
foreach
(array_keys(getColumnValidator()) as
$col) {
    if (!in_array($col,
$header)) {
        throw new
Exception("Missing column: $col");
    }
}

// Map CSV indexes to
column names
$colMap =
array_flip($header);

$rowNum = 1; // header =
row 0
while (($row =
fgetcsv($handle)) !== false) {
    $rowNum++;
    $cleanRow = [];

    foreach
(getColumnValidator() as $col =>
$validator) {
        $value =
$row[$colMap[$col]] ?? null;
        if (!$validator($value))
        {
            throw new
Exception("Row $rowNum: Invalid
value in column '$col' → $value");
        }
        $cleanRow[$col] =
$value; // save validated value
    }

    $validRows[] =
$cleanRow; // push into final dataset
}
fclose($handle);

```

```

foreach ($validRows as
&$studentDetails) {
    $predictionResult =
json_decode(predictStatus($studentDetail
s), true);
    $studentDetails['status'] =
$predictionResult['prediction'] ??
'Unknown';

    $studentDetails['major_at_risk_factor'] =
$predictionResult['major_predictor'] ??
'Unknown';

    $studentDetails['created_by_email'] =
$_SESSION['current_user']->email;
}

return $validRows;
}

function processSingleRecord()
{
    // Check if all necessary values
are present
    $studentDetails = [];
    foreach (getColumnValidator()
as $col => $validator) {
        if ($col === 'birthdate') {
            $dateValue =
DateTime::createFromFormat('Y-m-d',
$_POST[$col]);
            if ($dateValue) {
                $_POST[$col] =
$dateValue->format('d/m/Y');
            }
        }
        if (empty($_POST[$col]))
        { throw new
Exception("Error from value in input
field: $col");
        } elseif
(!$validator($_POST[$col])) {

```

```

        throw new
Exception("Error in {$col}.
Value:$_POST[$col] ");
    } else {
        $studentDetails[$col] =
$_POST[$col];
    }
}

$predictionResult =
json_decode(predictStatus($studentDetail
s), true);
    $studentDetails['status'] =
$predictionResult['prediction'] ??
'Unknown';

$studentDetails['major_at_risk_factor'] =
$predictionResult['major_predictor'] ??
'Unknown';

$studentDetails['created_by_email'] =
$_SESSION['current_user']->email;

    return [$studentDetails];
}

if
($ _SERVER['REQUEST_METHOD']
=== 'POST' &&
isset($_SESSION['current_user'])) {
    // check if ano yung pinost if
file ba or single record
    try {
        $result =
(!empty($_FILES['batch_record']['name'
])) ? processBatchRecord() :
processSingleRecord();
        if
(!empty($_FILES['batch_record']['name'
])) {
            foreach ($result as
&$record) {
                $record['intervention']
= ($record['status'] === 'At-Risk') ?
"Intervention Required" : "No
Intervention Needed";

```

```

    }
}
require_once __DIR__ .
'./createLog.php';
// echo
"<script>console.log(" .
json_encode($result) . ");</script>";
    } catch (Exception $e)
    { echo $e;
    }
}
?>

<link rel="stylesheet"
href="/STYLES/createRecord.css">
<div class="create-main">

    <?php if (isset($result)): ?>
        <div class="message">
            <h3>Success!</h3>
            <p>The record(s) have
been successfully uploaded.</p>
        </div>
    <?php else: ?>
        <div class="message">
            <h3>Error!</h3>
            <p>The uploading of
record(s) failed.</p>
        </div>
    <?php endif; ?>

    <?php if (isset($result)): ?>

        <div class="table-
container">
            <table>
                <thead>
                    <tr>
                        <?php foreach
(array_keys($result[0]) as $header): ?>
                            <th><?=
htmlspecialchars($header) ?></th>
                        <?php endforeach;
?>
                    </tr>
                </thead>

```

```

        <tbody>
        <?php foreach
($result as $row): ?>
            <tr>
            <?php foreach
($row as $value): ?>
                <td><?=
htmlspecialchars($value) ?></td>
            <?php
endforeach; ?>
            </tr>
        <?php endforeach;
?>

        </tbody>
    </table>
</div>

    <?php elseif (isset($e)): ?>
        <div class="error-
container"><?= htmlspecialchars($e-
>getMessage()) ?></div>
    <?php endif; ?>

    <?php if (isset($result)): ?>
        <form method="POST"
action="/uploadStudent.php"
onsubmit="return confirm('Are you sure
you want to submit these records to the
database?')">
            <input type="hidden"
name="records" value="<?=
htmlspecialchars(json_encode($result))
?>">
            <button type="submit"
id="confirm-btn">Confirm</button>
        </form>
    <?php endif; ?>

    <a
href="/../mainpage.php?page=records">
Cancel</a>
</div>
*****
<?php

// Enable error reporting for debugging

```

```

error_reporting(E_ALL);
ini_set('display_errors', 1);

require_once __DIR__.
"../USERHANDLING/User.php";
require_once __DIR__.
"../predictStatus.php";
session_start();

function updateRecord($student_details,
$record)
{
    $update_fields = []; // stores fields
like "student_name = ?" for the query
    $update_values = []; // stores values
for the fields like "Juan DC"
    $types = "ss"; // stores the appropriate
datatype for values

    foreach ($record as $key => $value) {
        // di kasama student_number cuz sa
dulo siya sa where clause di sa set
        if (isset($student_details[$key])
&& $student_details[$key] !== $value
&& $key !== 'student_number') {
            // appends the arrays and the
string above
            $update_fields[] = "$key = ?";
            $update_values[] =
$student_details[$key];

            // checks if its a gwa, then
append d for double (float) in types
            // else append s for string
            if (
                $key == "first_year_first_sem"
            ||
                $key ==
"first_year_second_sem" ||
                $key ==
"second_year_first_sem" ||
                $key ==
"second_year_second_sem"
            ) {
                $types .= "d";
            }
        }
    }
}

```

```

        } else {
            $types .= "s";
        }
    }
}

if ($update_values !== []) {

    // setting $student_details to
    $predictors cuz need ng certain format
    ng birthdate for prediction without
    // changing yung format ng for
    uploading sa database
    $predictors = $student_details;
    $dateValue =
    DateTime::createFromFormat('Y-m-d',
    $predictors['birthdate']);
    $predictors['birthdate'] =
    $dateValue->format('d/m/Y');
    $result =
    json_decode(predictStatus($predictors),
    true);

    $status = $result['prediction'] ??
    'Unknown';
    $major_predictor =
    $result['major_predictor'] ?? 'Unknown';

    $update_values[] =
    $record['student_number']; // adds the
    $student_number for the last ? in the
    WHERE
    $types .= 'i'; // adds the integer
    datatype for student_number
    $query = "UPDATE students SET
    major_at_risk_factor = ?, status = ?, " .
    implode(" ", $update_fields) . "
    WHERE student_number = ?";

    $stmt = getDbConnection()-
    >prepare($query);
    $stmt->bind_param($types,
    $major_predictor, $status,
    ...$update_values);
    $stmt->execute();
    $stmt->close();

```

```

        require_once "./createLog.php";
        createLog("Updated
        {$record['student_number']}: " .
        implode(" ", array_map(function
        ($field) {
            return str_replace(" = ?", "",
            $field);
        }, $update_fields)));
        return true;
    } else {
        return false;
    }
}

```

```

if ($_SERVER['REQUEST_METHOD']
=== 'POST' &&
isset($_SESSION['current_user'])) {

    require_once __DIR__ .
    "../DB/getDbConnection.php";

```

```

    $student_number =
    $_POST['student_number'];

    $stmt = getDbConnection()-
    >prepare("SELECT * FROM students
    WHERE student_number = ?");
    $stmt->bind_param("s",
    $student_number);
    $stmt->execute();
    $result = $stmt->get_result();

    // gets the record from the database
    using student_number
    $record = $result->fetch_assoc();

    if (!$record) {
        echo 'Student not found.';
    }

    $stmt->close();

    if (count($_POST) > 1) {

```

```
// means that may nakapost na other
details aside sa student_number.
```

```
// therefore nagsubmit gamit yung
from sa baba
```

```
// therefore may attempt to edit the
record of the student.
```

```
// here, we're passing yung $_POST
which contains the changes,
```

```
// and yung $record na galing db
if (updateRecord($_POST,
$record)) {
```

```
// Show success message and
redirect
```

```
echo "<script>alert('Student
record updated
successfully!');</script>";
```

```
// bale need 'to kasi ang
nanngyayari is this:
```

```
// update from user
```

```
// get from db the details na di pa
updated para mapakita sa form
```

```
// saka pa lang uupdate yung db.
```

```
// di puwede unahin update cuz it
cross checks muna if may nabago sa db
```

```
// if wala, edi di na need
magupdate
```

```
// since read muna before update,
// lumalabas sa form ehh yung
unupdated details
```

```
// kaya when you run it without
the script sa baba,
```

```
// uupdate mo tapos yung
previous text lalabas
```

```
// tapos update mo without
changing anything, lalabas yung
inupdate mo previously
```

```
// tapos next update yung
inupdate mo previously ulit
```

```
// e.g. current = 'shampoo' -
inupdate mo to 'shampo'-> current =
'shampoo'
```

```
// - update mo without changing -
> current = 'shampo'
```

```
echo "<script>
window.location.href =
'../mainpage.php?page=records';
</script>";
```

```
}
}
}
```

```
?>
```

```
<link rel="stylesheet"
href="/STYLES/updateRecord.css">
<div id="update-form">
```

```
<form method="post" action="<?php
echo
```

```
htmlspecialchars($_SERVER['REQUEST_
URI']); ?>" onkeydown="return
event.key != 'Enter';">
```

```
<fieldset id="student-details">
```

```
<legend>Student
Details</legend>
```

```
<label
id="student_number">Student Number:
<input type="text"
name="student_number" value="<?php
echo
```

```
htmlspecialchars($record['student_num
ber'] ?? ""); ?>" required></label>
```

```
<label id="ay_started">
Academic Year Started:
```

```
<select name="ay_started"
required>
```

```
<?php
$startYear = 2019;
$endYear = date('Y') + 1;
for ($year = $startYear;
$year < $endYear; $year++) {
    $nextYear = $year + 1;
    $yearString = "{$year}-
{$nextYear}";
```

```

                $selected =
($record['ay_started'] ?? "") ===
$yearString ? 'selected' : ";
                echo "<option
value=\"{" . $yearString . "\"
{" . $selected . ">{" . $yearString . "></option>";
                }
                ?>
                </select>
            </label>
            <label
id="student_name">Student Name:
<input type="text"
name="student_name" value="<?php
echo
htmlspecialchars($record['student_name']
) ?? " ); ?>" required></label>
            <label id="birthdate">Birthdate:
<input type="date" name="birthdate"
value="<?php echo
htmlspecialchars($record['birthdate'] ??
"); ?>" required></label>
            <label id="gender">Gender:
            <select name="gender"
required>
                <option
value="">Select</option>
                <option value="male"
<?php echo ($record['gender'] ?? " ) ===
'male' ? 'selected' : "; ?>>Male</option>
                <option value="female"
<?php echo ($record['gender'] ?? " ) ===
'female' ? 'selected' : ";
                ?>>Female</option>
                </select>
            </label>
            <label
id="citizenship">Citizenship: <input
type="text" name="citizenship"
value="<?php echo
htmlspecialchars($record['citizenship']
?? " ); ?>" required></label>
            <label id="religion">Religion:
<input type="text" name="religion"
value="<?php echo

```

```

htmlspecialchars($record['religion'] ??
"); ?>" required></label>
            <label
id="home_address">Home Address:
<input type="text"
name="home_address" value="<?php
echo
htmlspecialchars($record['home_address']
) ?? " ); ?>" required></label>

            <label id="course_id">Course:
            <select name="course_id"
required>
                <option
value="">Select</option>
                <?php
                require_once __DIR__ .
"/../DB/getDbConnection.php";
                $stmt = getDbConnection()-
>prepare("SELECT course_id,
course_name, college_id FROM
courses" .

((empty($_SESSION['current_user']-
>college_id)) ? "" :
                " WHERE college_id =
'{" . $_SESSION['current_user']-
>college_id . "'");
                $stmt->execute();
                $result = $stmt-
>get_result();
                if ($result) {
                    while ($row = $result-
>fetch_assoc()) {
                        $selected =
($record['course_id'] ?? " ) ===
$row['course_id'] ? 'selected' : ";
                        echo "<option
value=\"{" . $row['course_id'] . "\"
{" . $selected . ">{" . $row['course_id'] . " -
{" . $row['course_name'] . "></option>";
                    }
                    $result->free();
                }
                ?>
            </select>

```

```

        </label>
    </fieldset>
    <fieldset id="grades">
        <legend>Grades</legend>
        <label>First Year First Sem -
Major GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_first_sem_major"
value="<?php echo
htmlspecialchars($record['first_year_firs
t_sem_major'] ?? "); ?>" required>
            </label>
        <label>First Year First Sem - GE
GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_first_sem_ge"
value="<?php echo
htmlspecialchars($record['first_year_firs
t_sem_ge'] ?? "); ?>" required>
            </label>

        <label>First Year Second Sem -
Major GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_second_sem_major"
value="<?php echo
htmlspecialchars($record['first_year_sec
ond_sem_major'] ?? "); ?>" required>
            </label>
        <label>First Year Second Sem -
GE GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_second_sem_ge"
value="<?php echo
htmlspecialchars($record['first_year_sec
ond_sem_ge'] ?? "); ?>" required>
            </label>

```

```

        <label>Second Year First Sem -
Major GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_first_sem_major"
value="<?php echo
htmlspecialchars($record['second_year_f
irst_sem_major'] ?? "); ?>" required>
            </label>
        <label>Second Year First Sem -
GE GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_first_sem_ge"
value="<?php echo
htmlspecialchars($record['second_year_f
irst_sem_ge'] ?? "); ?>" required>
            </label>

        <label>Second Year Second Sem
- Major GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_second_sem_major"
value="<?php echo
htmlspecialchars($record['second_year_
second_sem_major'] ?? "); ?>" required>
            </label>
        <label>Second Year Second Sem
- GE GWA:
            <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_second_sem_ge"
value="<?php echo
htmlspecialchars($record['second_year_
second_sem_ge'] ?? "); ?>" required>
            </label>
        </fieldset>
    <div>
        <input type="submit"
value="Confirm">

```

```

        <button type="button"
onclick="window.location.href='../main
page.php?page=records'">Cancel</butto
n>

```

```

        </div>
    </form>
</div>
*****
<?php

```

```

// Enable error reporting for debugging
error_reporting(E_ALL);
ini_set('display_errors', 1);

```

```

require_once __DIR__ .
'../USERHANDLING/User.php';
session_start();

```

```

if ($_SERVER['REQUEST_METHOD']
=== 'POST' &&
isset($_SESSION['current_user']) &&
isset($_POST['records'])) {

```

```

    $result =
json_decode($_POST['records'], true);

```

```

    // echo "<script>console.log(" .
json_encode($result) . ");</script>";
    try {
        require_once __DIR__ .
'../DB/getDbConnection.php';
        $db = getDbConnection();

```

```

        foreach ($result as $record) {
            $stmt = $db->prepare("INSERT
            INTO students (
                student_number, ay_started,
                student_name, birthdate, gender,
                citizenship, home_address, religion,
                course_id,
                first_year_first_sem_ge,
                first_year_second_sem_ge,
                second_year_first_sem_ge,
                second_year_second_sem_ge,

```

```

                first_year_first_sem_major,
                first_year_second_sem_major,
                second_year_first_sem_major,
                second_year_second_sem_major,
                status, major_at_risk_factor,
                created_by_email
            ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
            ?, ?, ?, ?, ?, ?, ?, ?, ?));

```

```

            $record['birthdate'] = date('Y-m-
            d', strtotime(str_replace('-', '/',
            $record['birthdate'])));

```

```

            $stmt-
            >bind_param( "dssssssssdddddd
            dddsss",
                $record['student_number'],
                $record['ay_started'],
                $record['student_name'],
                $record['birthdate'],
                $record['gender'],
                $record['citizenship'],
                $record['home_address'],
                $record['religion'],
                $record['course_id'],

```

```

            $record['first_year_first_sem_ge'],

```

```

            $record['first_year_second_sem_ge'],

```

```

            $record['second_year_first_sem_ge'],

```

```

            $record['second_year_second_sem_ge'],

```

```

            $record['first_year_first_sem_major'],

```

```

            $record['first_year_second_sem_major'],

```

```

            $record['second_year_first_sem_major'],

```

```

            $record['second_year_second_sem_majo
            r'],

```

```

                $record['status'],

```

```

            $record['major_at_risk_factor'],

```

```

                $record['created_by_email'],

```

```

            );

```



```

        $stmt->execute();
        $stmt->close();
    }
} catch (Exception $e) {
    // die("Error: " . $e->getMessage());
}

// echo "uploaded";
}

$db->close();
header('Location:
../mainpage.php?page=records');
exit();
*****

<?php

if (!isset($_SESSION['current_user']))
    { header("Location:
../USERHANDLING/logout.php");
      exit();
    }

//
// QUERY FOR THE NUMBERS
//

require_once __DIR__ .
'../DB/getDbConnection.php';
$conn = getDbConnection();
// Query to get the count of students
// Query to get the total count of students
$sql = "SELECT
        (SELECT COUNT(*) FROM
students) AS total_students,
        (SELECT COUNT(*) FROM
students WHERE status = 'On-Time')
AS on_time_students,
        (SELECT COUNT(*) FROM
students WHERE status = 'At-Risk') AS
at_risk_students";
$result = $conn->query($sql);

// Fetch the counts

$total_students = $on_time_students =
$at_risk_students =
$percentage_On_Time = 0;
if ($result && $row = $result-
>fetch_assoc()) {
    $total_students =
$row['total_students'];
    $on_time_students =
$row['on_time_students'];
    $at_risk_students =
$row['at_risk_students'];
    $percentage_On_Time =
$total_students > 0 ? ($on_time_students
/ $total_students) * 100 : 0;
}

//
// END
//

?>

<link rel="stylesheet"
href="/STYLES/dashboard.css">

<div id="filter">
    <!-- Filter options will go here -->
</div>
<div id="numbers">
    <div id="active-students">
        <h2><?php echo $total_students;
?></h2>
        <p>Active Students</p>
    </div>
    <div id="on-time-count">
        <h2><?php echo
$on_time_students; ?></h2>
        <p>On-Time Students</p>
    </div>
    <div id="at-risk-count">
        <h2><?php echo $at_risk_students;
?></h2>
        <p>At-Risk Students</p>
    </div>
    <div id="on-time-percent">

```

```

        <h2><?php echo
(int)$percentage_On_Time; ?>%</h2>
        <p>On-Time Progression</p>
    </div>
</div>

<div id="bar">
    <canvas id="barchart-age-risk">
        <!-- Bar chart 1 will render here -->
    </canvas>
    <canvas id="barchart-district-risk">
        <!-- Bar chart 2 will render here -->
    </canvas>
    <canvas id="barchart-gender-risk">
        <!-- Bar chart 3 will render here -->
    </canvas>
</div>
<canvas id="linechart-year">
    <!-- Line chart will render here -->
</canvas>

<?php

//
// query for gender based risk
//

require_once __DIR__ .
'../DB/getDbConnection.php';
$conn = getDbConnection();

// Query to fetch gender-wise data
$sql = "SELECT
        gender,
        SUM(CASE WHEN status =
'On-Time' THEN 1 ELSE 0 END) AS
on_time,
        SUM(CASE WHEN status =
'At-Risk' THEN 1 ELSE 0 END) AS
at_risk
        FROM students
        GROUP BY gender";

$result = $conn->query($sql);

```

```

$genderData = [];

if ($result) {
    while ($row = $result->fetch_assoc())
    {
        $gender = $row['gender'];
        $genderData[$gender] =
            [ 'on_time' =>
(int)$row['on_time'],
            'at_risk' => (int)$row['at_risk']
            ];
    }
}

//
// query for age group based risk
//

$sql = "SELECT
        CASE
            WHEN
                TIMESTAMPDIFF( YE
                AR,
                birthdate,

                STR_TO_DATE(CONCAT(SUBSTRIN
G(ay_started, 1, 4), '-08-01'), '%Y-%m-
%d')
                ) +
                (

                DATE_FORMAT(STR_TO_DATE(CO
NCAT(SUBSTRING(ay_started, 1, 4), '-
08-01'), '%Y-%m-%d'), '%m%d')
                < DATE_FORMAT(birthdate,
'%m%d')
                ) <= 20.8
                THEN 'Typical Age Enrollee'
                ELSE 'Late Age Enrollee'
            END AS age_group,
        SUM(CASE WHEN status = 'On-
Time' THEN 1 ELSE 0 END) AS
on_time,
        SUM(CASE WHEN status = 'At-Risk'
THEN 1 ELSE 0 END) AS at_risk
        FROM students
        GROUP BY age_group;";

```

```

$result = $conn->query($sql);

$ageGroupData = [];

if ($result) {
    while ($row = $result->fetch_assoc())
    {
        $ageGroup = $row['age_group'];
        $ageGroupData[$ageGroup] =
            [ 'on_time' =>
(int)$row['on_time'],
            'at_risk' => (int)$row['at_risk']
            ];
    }
}

//
// query for district based risk
//

$sql = "SELECT
    CASE
        WHEN (LOWER(home_address)
LIKE '%malolos%' OR
LOWER(home_address) LIKE
'%bulakan%' OR
LOWER(home_address) LIKE
'%calumpit%'
        OR LOWER(home_address)
LIKE '%hagonoy%' OR
LOWER(home_address) LIKE
'%paombong%' OR
LOWER(home_address) LIKE
'%pulilan%')
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '1'
        WHEN (LOWER(home_address)
LIKE '%baliwag%' OR
LOWER(home_address) LIKE
'%bustos%' OR LOWER(home_address)
LIKE '%plaridel%')
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '2'
        WHEN (LOWER(home_address)
LIKE '%doña remedios trinidad%' OR
LOWER(home_address) LIKE '%san
ildefonso%'
        OR LOWER(home_address)
LIKE '%san miguel%' OR
LOWER(home_address) LIKE '%san
rafael%'
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '3'
        WHEN (LOWER(home_address)
LIKE '%meycauayan%' OR
LOWER(home_address) LIKE
'%marilao%' OR
LOWER(home_address) LIKE
'%obando%'
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '4'
        WHEN (LOWER(home_address)
LIKE '%balagtas%' OR
LOWER(home_address) LIKE
'%bocaue%' OR
LOWER(home_address) LIKE
'%guiguinto%'
        OR LOWER(home_address)
LIKE '%pandi%'
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '5'
        WHEN (LOWER(home_address)
LIKE '%angat%' OR
LOWER(home_address) LIKE
'%norzagaray%' OR
LOWER(home_address) LIKE '%sta.
maria%'
        OR LOWER(home_address)
LIKE '%san jose del monte%'
        AND LOWER(home_address)
LIKE '%bulacan%' THEN '6'
        ELSE 'Others'
        END AS district,
    SUM(CASE WHEN status = 'On-
Time' THEN 1 ELSE 0 END) AS
on_time,
    SUM(CASE WHEN status = 'At-Risk'
THEN 1 ELSE 0 END) AS at_risk
FROM students
GROUP BY district;
";

```

```

$result = $conn->query($sql);

$districtData = [];

if ($result) {
    while ($row = $result->fetch_assoc())
    {
        $district = $row['district'];
        $districtData[$district] =
            [ 'on_time' =>
(int)$row['on_time'],
            'at_risk' => (int)$row['at_risk']
            ];
    }
}

//
// query for ay_started risk
//
$sql = "SELECT
#SUM(CASE WHEN status = 'On-Time'
THEN 1 ELSE 0 END) AS on_time,
SUM(CASE WHEN status = 'At-Risk'
THEN 1 ELSE 0 END) AS at_risk,
ay_started
FROM students GROUP BY ay_started
ORDER BY ay_started;
";

$result = $conn->query($sql);

$yearData = [];

if ($result) {
    while ($row = $result->fetch_assoc())
    {
        $year = $row['ay_started'];
        $yearData[$year] =
            [ '#on_time' =>
(int)$row['on_time'],
            'at_risk' => (int)$row['at_risk']
            ];
    }
}

```

```

?><script
src="https://cdn.jsdelivr.net/npm/chart.js
"></script>
<script
src="https://cdn.jsdelivr.net/npm/chartjs-
plugin-zoom@2.0.1"></script>
<script
src="https://cdn.jsdelivr.net/npm/chartjs-
plugin-datalabels@2"></script>

<script>
    Chart.register(ChartZoom);

Chart.unregister(Chart.registry.getPlugin
('legend'));
    Chart.defaults.color = 'black';

    let charts = {};

    function renderChart(id, type, data,
options) {
        const ctx =
document.getElementById(id);
        if (charts[id]) charts[id].destroy();

        charts[id] = new Chart(ctx,
        { type,
        data,
        options: {
            responsive: false,
            animation: false,
            maintainAspectRatio: true,

            scales:
                { x: {
                    beginAtZero: true
                },
                y: {
                    beginAtZero: true
                }
            },
            plugins:
                { zoom:
                    {
                        zoom: {
                            mode: 'x',
                            onZoom: ({

```

```

        chart
    })=>
chart.update('none')
    },
    pan: {
        enabled: true,
        mode: 'x'
    }
    },
    ...options.plugins,
    legend: {
        display: false
    },
    },
    ...options
    }
    });
}

renderChart('barchart-gender-risk',
'bar', {
    labels: <?php echo
json_encode(array_keys($genderData));
?>,
    datasets: [{
        label: 'On-Time',
        data: <?php echo
json_encode(array_column($genderData
, 'on_time')); ?>,
        backgroundColor: '#bd4848ff',
    },
    {
        label: 'At-Risk',
        data: <?php echo
json_encode(array_column($genderData
, 'at_risk')); ?>,
        backgroundColor: '#600606',
    }
    ], {
    plugins:
    { title:
    {
        display: true,
        text: 'Age-wise At-Risk and
On-Time Students'
    }
    },
    });
renderChart('barchart-district-risk',
'bar', {
    labels: <?php echo
json_encode(array_keys($districtData));
?>,
    datasets: [{
        label: 'On-Time',

```

```

        text: 'Gender-wise At-Risk and
On-Time Students'
    }
    },
    });

renderChart('barchart-age-risk', 'bar',
    { labels: <?php echo
json_encode(array_keys($ageGroupData
)); ?>,
    datasets: [{
        label: 'On-Time',
        data: <?php echo
json_encode(array_column($ageGroupD
ata, 'on_time')); ?>,
        backgroundColor: '#bd4848ff',
    },
    {
        label: 'At-Risk',
        data: <?php echo
json_encode(array_column($ageGroupD
ata, 'at_risk')); ?>,
        backgroundColor: '#600606',
    }
    ], {
    }, {
    plugins:
    { title:
    {
        display: true,
        text: 'Age-wise At-Risk and
On-Time Students'
    }
    },
    });
renderChart('barchart-district-risk',
'bar', {
    labels: <?php echo
json_encode(array_keys($districtData));
?>,
    datasets: [{
        label: 'On-Time',

```

```

        data: <?php echo
json_encode(array_column($districtData
, 'on_time')); ?>,
        backgroundColor: '#bd4848ff',
    },
    {
        label: 'At-Risk',
        data: <?php echo
json_encode(array_column($districtData
, 'at_risk')); ?>,
        backgroundColor: '#600606',
        fill: false
    }
],
}, {
    plugins:
    { title:
    {
        display: true,
        text: 'District-wise At-Risk and
On-Time Students'
    }
    },
    indexAxis: 'y', // This makes the
chart horizontal
    scales:
    { x: {
        beginAtZero: true
    },
    y: {
        ticks: {
            autoSkip: false
        }
    }
    }
});

renderChart('linechart-year', 'line',
    { labels: <?php echo
json_encode(array_keys($yearData));
?>,
    datasets: [{
        label: 'On-Time',
        data: <?php echo
json_encode(array_column($yearData,
'on_time')); ?>,
        backgroundColor: '#bd4848ff',
        borderColor: '#bd4848ff',
        fill: false
    },
    {
        label: 'At-Risk',
        data: <?php echo
json_encode(array_column($yearData,
'at_risk')); ?>,
        backgroundColor: '#600606',
        borderColor: '#600606',
        fill: false
    }
],
}, {
    plugins:
    { title:
    {
        display: true,
        text: 'Year-wise At-Risk
Students'
    }
    },
    scales:
    { y: {
        beginAtZero: true,
        grid: {
            drawOnChartArea: false
        },
        ticks: {
            padding: 0
        }
    }
    },
    layout:
    { padding:
    0
    },
    maintainAspectRatio: true,
    aspectRatio: 4
});
</script>
*****
<?php
if (!isset($_SESSION['current_user'])) {

```

```

    header("Location:
    ../../USERHANDLING/logout.php");
    exit();
}

if ($_SERVER['REQUEST_METHOD']
=== 'POST') {
    require_once __DIR__ .
    "../../FUNCTIONS/deleteArchiveStudent.
    php";
    if ($_POST['function'] === 'delete' ||
    $_POST['function'] === 'archive') {

        deleteArchiveStudent($_POST['student_
        number'], $_POST['function']);
    }
}
?>

<link rel="stylesheet"
href="/STYLES/records.css">
<div class="records-header">
    <h3>Student Records</h3>
    <hr>
    <div class="create-filter-container">
        <form class="add-student">
            <h4>Add Student</h4>
            <button
            id="openStudentFormModal"
            type="button">Input</button>
        </form>

        <form class="upload-file"
        method="post"
        action="/FUNCTIONS/predictStudent.p
        hp" enctype="multipart/form-data">
            <h4>Import Students
            (CSV)</h4>
            <br>
            <div>
                <input type="file"
                name="batch_record" accept=".csv"
                required>
                <input type="submit"
                value="Import">
            </div>

```

```

    </form>
    <form class="filter-records"
    method="get">
        <h4>Filter Records</h4>
        <input type="hidden"
        name="page" value="records">
        <div>
            <select
            name="filter_academic_year">
                <option value="">A.Y.
                Started</option>
                <?php
                $startYear = 2010;
                $endYear = date('Y') + 1;
                for ($year = $startYear;
                $year < $endYear; $year++) {
                    $nextYear = $year + 1;
                    $selected =
                    (isset($_GET['filter_academic_year'])
                    && $_GET['filter_academic_year'] ==
                    "{$year}-{$nextYear}") ? 'selected' : "";
                    echo "<option
                    value=\"{$year}-{$nextYear}\"
                    $selected>{$year}-
                    {$nextYear}</option>";
                }
            </select>
            <select
            name="filter_course_id">
                <option
                value="">Course</option>
            <?php

                require_once __DIR__ .
                "../../DB/getDbConnection.php";
                $conn =
                getDbConnection();
                $stmt = $conn-
                >prepare("SELECT course_id,
                college_id FROM courses" .

                ((empty($_SESSION['current_user'])-
                >college_id)) ? "" :
                " WHERE college_id =
                ?");

```

```

        if
(!empty($_SESSION['current_user']-
>college_id)) {
            $stmt->bind_param("s",
$_SESSION['current_user']-
>college_id);
            }
            $stmt->execute();

            $result = $stmt-
>get_result();
            if ($result) {
                while ($row = $result-
>fetch_assoc()) {
                    $selected =
(isset($_GET['filter_course_id']) &&
$_GET['filter_course_id'] ==
$row['course_id']) ? 'selected' : ";
                    echo "<option
value=\"{"$row['course_id']}\
$selected>{"$row['course_id']}</option>
";
                }
                $result->free();
            }
            $stmt->close();
            ?>
</select>

<!-- Added Status Filter -->
<select name="filter_status">
    <option
value="">Status</option>
    <option value="On-Time"
<?php if (isset($_GET['filter_status'])
&& $_GET['filter_status'] == "On-
Time") echo 'selected'; ?>>On-
Time</option>
    <option value="At-Risk"
<?php if (isset($_GET['filter_status'])
&& $_GET['filter_status'] == "At-Risk")
echo 'selected'; ?>>At-Risk</option>
</select>

<input type="submit"
value="Filter">

```

```

</div>
</form>
</div>
</div>

<div id="student-form">
    <form method="post"
action="/FUNCTIONS/predictStudent.p
hp">
        <fieldset id="student-details">
            <legend>Student
Details</legend>

            <label
id="student_number">Student Number:
            <input type="text"
name="student_number"
required></label>
            <label id="ay_started">
                Academic Year Started:
                <select name="ay_started"
required>
                    <?php
                        $startYear = 2019;
                        $endYear = date('Y') + 1;
                        for ($year = $startYear;
$year < $endYear; $year++) {
                            $nextYear = $year + 1;
                            echo "<option
value=\"{"$year}-
{"$nextYear}\
">{"$year}-
{"$nextYear}</option>";
                        }
                    ?>
                </select>
            </label>
            <label
id="student_name">Student Name:
            <input type="text"
name="student_name"
required></label>
            <label id="birthdate">Birthdate:
            <input type="date" name="birthdate"
id="birthdate" required></label>
            <label id="gender">Gender:

```



```

        <select name="gender"
required>
        <option
value="">Select</option>
        <option
value="male">Male</option>
        <option
value="female">Female</option>
        </select>
    </label>
    <label
id="citizenship">Citizenship: <input
type="text" name="citizenship"
required></label>
    <label id="religion">Religion:
<input type="text" name="religion"
required></label>
    <label
id="home_address">Home Address:
<input type="text"
name="home_address"
required></label>

    <label id="course_id">Course:
    <select name="course_id"
required>
        <option
value="">Select</option>
        <?php

            require_once __DIR__ .
"/../DB/getDbConnection.php";
            $stmt = getDbConnection()-
>prepare("SELECT course_id,
course_name, college_id FROM
courses" .

((empty($_SESSION['current_user'])-
>college_id)) ? "" :
" WHERE college_id =
'{"$_SESSION['current_user']"-
>college_id}'");
            $stmt->execute();
            $result = $stmt-
>get_result();
            if ($result) {

```

```

                while ($row = $result-
>fetch_assoc()) {
                    echo "<option
value=\"{"$row['course_id']}\">>{"$row['c
ourse_id']}" -
{"$row['course_name']}</option>";
                }
                $result->free();
            }
        ?>
    </select>
    </label>
</fieldset>
<fieldset id="grades">
    <legend>Grades</legend>
    <label>First Year First Sem -
Major GWA:
        <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_first_sem_major"
required>
    </label>
    <label>First Year First Sem - GE
GWA:
        <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_first_sem_ge"
required>
    </label>

    <label>First Year Second Sem -
Major GWA:
        <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="first_year_second_sem_major"
required>
    </label>
    <label>First Year Second Sem -
GE GWA:
        <input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"

```

```

name="first_year_second_sem_ge"
required>
</label>

```

<label>Second Year First Sem - Major GWA:

```

<input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_first_sem_major"
required>
</label>

```

<label>Second Year First Sem - GE GWA:

```

<input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_first_sem_ge"
required>
</label>

```

<label>Second Year Second Sem - Major GWA:

```

<input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_second_sem_major"
required>
</label>

```

<label>Second Year Second Sem - GE GWA:

```

<input type="number"
min="0" max="5" step="0.01"
inputmode="decimal"
name="second_year_second_sem_ge"
required>
</label>

```

</fieldset>

<div>

```

<input type="submit"
value="Confirm">
<button type="button"
onclick="document.getElementById('student-form').style.display='none'">Cancel</button>

```

```

</div>
</form>
</div>

```

<script>

```

document.getElementById('openStudentFormModal').onclick = function() {
    document.getElementById('student-form').style.display = 'block';
};
document.getElementById('student-form').addEventListener('click',
function(event) {
    if (event.target === this)
        { this.style.display = 'none';
        }
});

```

// Close modal when clicking outside the modal content

```

window.onclick = function(event)
{ var modal =
document.getElementById('student-form');
if (event.target == modal)
{ modal.style.display = 'none';
}
};

```

</script>

<?php

```

require_once __DIR__ .
'../DB/getDbConnection.php';
$conn = getDbConnection();

```

```

$filters = [];

```

```

if
(empty($_GET['filter_academic_year'])
) {

```

```

    $filters[] = "s.ay_started = " . $conn->real_escape_string($_GET['filter_academic_year']) . " ";
}

```

```

if (!empty($_GET['filter_course_id'])) {

```

```

    $filters[] = "s.course_id = " . $conn-
>real_escape_string($_GET['filter_cours
e_id']) . """;
}
// Added filter for Status
if (!empty($_GET['filter_status'])) {
    $filters[] = "s.status = " . $conn-
>real_escape_string($_GET['filter_statu
s']) . """;
}

$query = "SELECT s.student_number,
s.ay_started, s.student_name, s.birthdate,
s.gender,
    s.citizenship,
s.home_address, s.religion, s.course_id,
    s.first_year_first_sem_major,
s.first_year_second_sem_major,
s.second_year_first_sem_major,
s.second_year_second_sem_major,
    s.first_year_first_sem_ge,
s.first_year_second_sem_ge,
s.second_year_first_sem_ge,
s.second_year_second_sem_ge,
    s.status, s.major_at_risk_factor,
s.created_by_email, s.created_at,
s.updated_at FROM students s";

if (empty($_SESSION['current_user']-
>college_id)) {
    if (!empty($filters)) {
        $query .= " WHERE " . implode("
AND ", $filters);
    }
} else {
    $college_id = $conn-
>real_escape_string($_SESSION['curren
t_user']->college_id);
    $query .= " INNER JOIN courses c
ON s.course_id = c.course_id
    WHERE c.college_id =
'{$college_id}' AND s.is_archived =
'n";
    if (!empty($filters)) {
        $query .= " AND " . implode("
AND ", $filters);

```

```

    }
}

$stmt = $conn->prepare($query);
$stmt->execute();
$result = $stmt->get_result();

if ($result && $result->num_rows > 0)
{ echo '<table class="student-
records">';
    echo '<tr>
        <th>Student Number</th>
        <th>A.Y. Started</th>
        <th>Student Name</th>
        <th>Course</th>
        <th>1st Yr 1st Sem</th>
        <th>1st Yr 2nd Sem</th>
        <th>2nd Yr 1st Sem</th>
        <th>2nd Yr 2nd Sem</th>
        <th>Status</th>
        <th>Major At-Risk Factor</th>
        <th>Intervention</th>
        <th>Actions</th>
    </tr>';
    while ($row = $result->fetch_assoc())
    {
        echo '<tr class="main-table-row">';
        echo '<td>'.
htmlspecialchars($row['student_number'
]) . '</td>';
        echo '<td>'.
htmlspecialchars($row['ay_started']) .
'</td>';
        echo '<td>'.
htmlspecialchars($row['student_name'])
. '</td>';

        echo '<td>'.
htmlspecialchars($row['course_id']) .
'</td>';

        echo '<td>'. 'Major GWA:<br>' .
htmlspecialchars($row['first_year_first_s
em_major']) . "<br><br>GE
GWA:<br>" .

```

```

htmlspecialchars($row['first_year_first_s
em_ge']) . '</td>';
    echo '<td>' . 'Major GWA:<br>' .
htmlspecialchars($row['first_year_secon
d_sem_major']) . "<br><br>GE
GWA:<br>" .
htmlspecialchars($row['first_year_secon
d_sem_ge']) . '</td>';
    echo '<td>' . 'Major GWA:<br>' .
htmlspecialchars($row['second_year_firs
t_sem_major']) . "<br><br>GE
GWA:<br>" .
htmlspecialchars($row['second_year_firs
t_sem_ge']) . '</td>';
    echo '<td>' . 'Major GWA:<br>' .
htmlspecialchars($row['second_year_sec
ond_sem_major']) . "<br><br>GE
GWA:<br>" .
htmlspecialchars($row['second_year_sec
ond_sem_ge']) . '</td>';

    echo '<td>' .
htmlspecialchars($row['status']) . '</td>';
    echo '<td>' .
htmlspecialchars($row['major_at_risk_fa
ctor']) . '</td>';
    echo '<td>' .
((htmlspecialchars($row['status']) ==
"On-Time") ? "No Intervention Needed"
: "Intervention Required") . '</td>';

    echo '<td>';

    /* ARCHIVE BUTTON – gold
theme */
    echo '<form method="post"
action="" .
htmlspecialchars($_SERVER["REQUE
ST_URI"], ENT_QUOTES) . "">
        <input type="hidden"
name="function" value="archive">
        <input type="hidden"
name="student_number" value="" .
htmlspecialchars($row["student_number
"], ENT_QUOTES) . "">

```

```

        <button class="record-action"
id="archive" type="submit"
onclick="return confirm('\Are you sure
you want to archive this
student?\'')">Archive</button>

```

```
</form>;
```

```

/* ADMIN-ONLY BUTTONS */
if ($_SESSION['current_user']-
>role == 'admin') {

```

```

    /* UPDATE BUTTON */
    echo '<form method="post"
action=" ../FUNCTIONS/updateStudent.
php">

```

```

        <input type="hidden"
name="student_number" value="" .
htmlspecialchars($row["student_number
"], ENT_QUOTES) . "">

```

```

        <button type="submit"
class="record-action"
id="update">Update</button>
    </form>;

```

```

    /* DELETE BUTTON */
    echo '<form method="post"
action="" .
htmlspecialchars($_SERVER["REQUE
ST_URI"], ENT_QUOTES) . "">

```

```

        <input type="hidden"
name="function" value="delete">
        <input type="hidden"
name="student_number" value="" .
htmlspecialchars($row["student_number
"], ENT_QUOTES) . "">

```

```

        <button class="record-action"
id="delete" type="submit"
onclick="return confirm('\Are you sure
you want to delete this student?\'')">

```

```

            Delete
        </button>

```

```

    </form>
    ;

```

```
    }
```

```

        echo '</td>';

        echo '</tr>';
        echo '<tr class="secondary-table-
row">';
        echo '
        <td colspan="10">
            <div class="secondary-table-row-
details">
                <div>
                    <p>Birthdate: ' .
htmlspecialchars($row['birthdate']) .
'</p>
                    <p>Gender: ' .
htmlspecialchars($row['gender']) . '</p>
                    <p>Citizenship: ' .
htmlspecialchars($row['citizenship']) .
'</p>
                    <p>Home Address: ' .
htmlspecialchars($row['home_address'])
. '</p>
                    <p>Religion: ' .
htmlspecialchars($row['religion']) . '</p>
                    <p>Created By Email: ' .
htmlspecialchars($row['created_by_email']) . '</p>
                    <p>Created At: ' .
htmlspecialchars($row['created_at']) .
'</p>
                    <p>Updated At: ' .
htmlspecialchars($row['updated_at']) .
'</p>
                </div>
            </div>
        </td>
';

        echo '</tr>';
    }

    echo '</table>';
    $result->free();
} else {

```

```

        echo '<p>No student records
found.</p>';
    }
?>

<script>

document.addEventListener("DOMCont
entLoaded", () => {
    document.querySelectorAll(".main-
table-row").forEach(row => {
        row.addEventListener("click",
(event) => {
            // ignore clicks on buttons (or
inside buttons)
            if
(event.target.closest("button")) {
                return;
            }

            const secondaryRow =
row.nextElementSibling;
            if (secondaryRow &&
secondaryRow.classList.contains("secon
dary-table-row")) {
                const detailsDiv =
secondaryRow.querySelector(".secondar
y-table-row-details");
                if (detailsDiv) {

                    detailsDiv.classList.toggle("show");
                }
            }
        });
    });
});
</script>
*****
<?php
require_once
'./USERHANDLING/login.php';
$error = "";
if ($_SERVER['REQUEST_METHOD']
=== 'POST') {
    $email = trim($_POST['email'] ?? "");

```

```

$password =
trim($_POST['password'] ?? "");
if ($email && $password) {
    if (login($email, $password))
        { header('Location:
../mainpage.php?page=dashboard');
        exit();
        }
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <title>BulSU Student Prediction
System</title>
    <link rel="icon" type="image/png"
href="/IMAGE/logo.png">
    <link rel="stylesheet"
href="/STYLES/login.css">
</head>
<body>
<div class="login-container">
    <div class="login-illustration">
        
    </div>

    <div class="login-form">
        <h2>Log In</h2>
        <p>Welcome back! Please login to
your account.</p>

```

```

<form method="post" action="">
    <div class="input-group">
        <input type="text" name="email"
placeholder="Enter your Email"
required>
    </div>
    <div class="input-group">
        <input type="password"
name="password" id="password"
placeholder="Enter your Password"
required>
    </div>
    <div class="options">
        <input type="checkbox"
id="showPassword"
onclick="document.getElementById('pa
ssword').type = this.checked ? 'text' :
'password';">
        <label for="showPassword">Show
Password</label>
    </div>
    <button
type="submit">LOGIN</button>
</form>
</div>
<script>
    function togglePassword()
    { const pass =
document.getElementById("password");
        pass.type = pass.type ===
"password" ? "text" : "password";
    }
</script>
</body>
</html>

```

